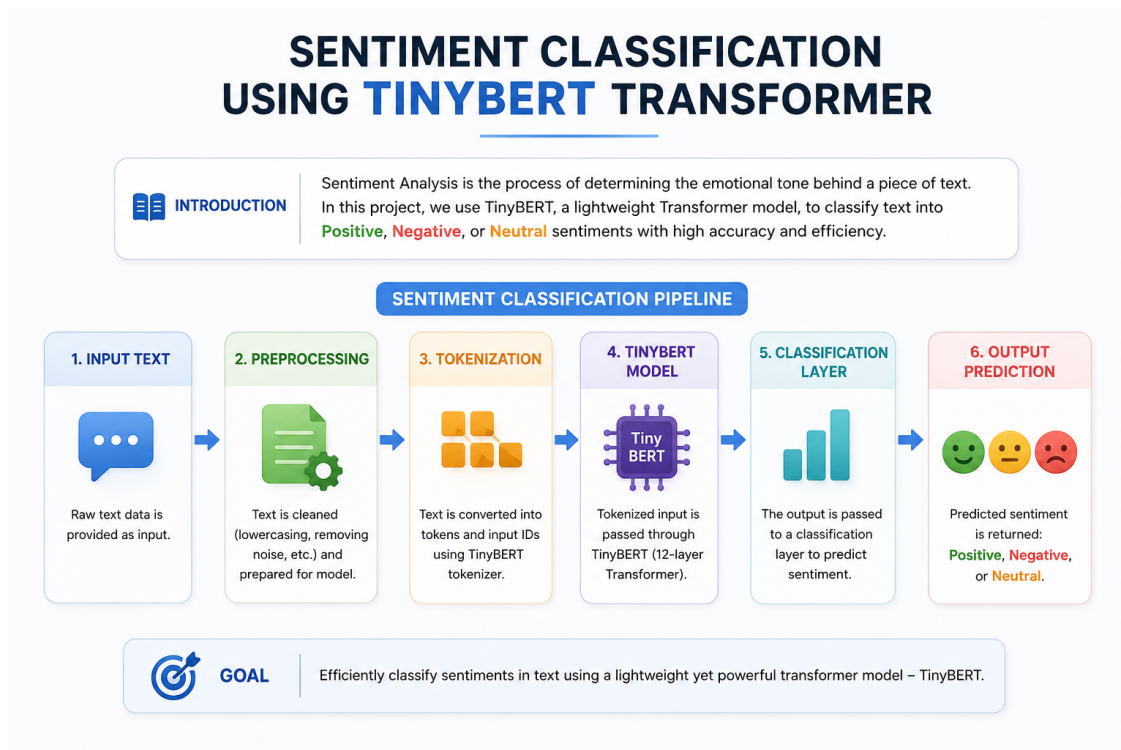


Sentiment Classification Using TinyBERT Transformer

August 5, 2026

1 Sentiment Classification Using TinyBERT Transformer



2 Load Data

2.1 Load Data With Hugging Face Dataset Library

```
[1]: import warnings
     warnings.filterwarnings('ignore')
```

```
[2]: import pandas as pd

     data = pd.read_csv('https://raw.githubusercontent.com/laxmimerit/
     ↪All-CSV-ML-Data-Files-Download/refs/heads/master/IMDB-Dataset.csv')
     data.head()
```

```
[2]:                                     review sentiment
0 One of the other reviewers has mentioned that ... positive
1 A wonderful little production. <br /><br />The... positive
2 I thought this was a wonderful way to spend ti... positive
3 Basically there's a family where a little boy ... negative
4 Petter Mattei's "Love in the Time of Money" is... positive
```

```
[3]: # !pip install datasets
```

```
[4]: from datasets import Dataset
```

```
[5]: dataset = Dataset.from_pandas(data)
dataset = dataset.train_test_split(test_size=0.3)
dataset
```

```
[5]: DatasetDict({
  train: Dataset({
    features: ['review', 'sentiment'],
    num_rows: 35000
  })
  test: Dataset({
    features: ['review', 'sentiment'],
    num_rows: 15000
  })
})
```

```
[6]: # input_ids, attention_mask, label --> number

data['sentiment'].value_counts()
```

```
[6]: sentiment
positive    25000
negative    25000
Name: count, dtype: int64
```

```
[7]: label2id = {'negative':0, 'positive':1}
id2label = {0:'negative', 1:'positive'}

dataset = dataset.map(lambda x: {'label':label2id[x['sentiment']]})
```

```
Map: 100%|          | 35000/35000 [00:07<00:00, 4480.34 examples/s]
Map: 100%|          | 15000/15000 [00:05<00:00, 2908.32 examples/s]
```

```
[8]: dataset
```

```
[8]: DatasetDict({
  train: Dataset({
    features: ['review', 'sentiment', 'label'],
```

```

        num_rows: 35000
    })
    test: Dataset({
        features: ['review', 'sentiment', 'label'],
        num_rows: 15000
    })
})

```

```
[9]: dataset['train'][0]
```

```
[9]: {'review': "I always follow the Dakar, so when my husband bought Charlie's 'Race to Dakar' DVD home I couldn't wait to watch it! Of course we'd seen the broadcast of the race when the actual race was on, but that never gives the background and specific teams.<br /><br />If you watched Long Way Round then you won't be surprised by the language which frankly I find more amusing than offensive.<br /><br />I think the only thing that annoyed me about the DVD was Charlie's hair, but he had it styled before Dakar so my feminine need for neatness was assuaged; tho' I could have lived without the 'flame' undies lol As with LWR, the preparation was every bit as interesting as the race itself. I nearly cried when Charlie broke his hand, and winced at every bruise he sustained while training...and of course the death of Andy Caldicott...that was an appalling tragedy, but then every year there's something.<br /><br />Russ drives me nuts, although his attitude has improved a thousand times from the argumentative cynic he was in LWR. It's great to see him get along so well now with Charlie.<br /><br />What I learned from this odyssey was - 1. never let Scorpion prepare your vehicle for ANYTHING! - they had months to prepare the X5, and still the day before the team left for Lisbon, Scorpion had only done half of things that needed to be done, and the vehicle was a pain throughout the whole race; 2. the Dakar organizers need to put a lot more work into their rider/driver retrieval plan - leaving Matt (and presumably a large number of other riders/drivers out to dry the way they did was nothing short of culpable negligence; 3. Charlie has an endearing enthusiasm for 'rough and tough' adventure but needs to toughen up a lot to really perform as he'd like; and finally, 4. Charlie and Ewan are planning another of these epos called the Long Way Down in 2007, and I can't wait to get my hands on it! :D If you love bikes and/or genuinely nice blokes 'having a go', you have to watch this, I guarantee you love it. It's very entertaining.<br /><br />In conclusion, to Simon Pavey - you sir are a hero, I was so impressed by the your 'quiet achiever' manner and the fact that you actually finished...just incredible considering what an monumentally difficult race it is. And to Charlie, Matt and the rest of the team - full marks for pulling it off. To think that a relatively green team could have achieved so much is truly admirable. You're all wonderful.",
'sentiment': 'positive',
'label': 1}
```

3 Data Tokenization

```
[10]: from transformers import AutoTokenizer
import torch
```

```
[11]: device = torch.device('cuda') if torch.cuda.is_available() else torch.
      ↪device('cpu')
print(device)

model_ckpt = 'huawei-noah/TinyBERT_General_4L_312D'
tokenizer = AutoTokenizer.from_pretrained(model_ckpt, use_fast = True)
```

cuda

```
[12]: tokenizer(dataset['train'][0]['review'])
```

```
[12]: {'input_ids': [101, 1045, 2467, 3582, 1996, 4830, 6673, 1010, 2061, 2043, 2026,
3129, 4149, 4918, 1005, 1055, 1005, 2679, 2000, 4830, 6673, 1005, 4966, 2188,
1045, 2481, 1005, 1056, 3524, 2000, 3422, 2009, 999, 1997, 2607, 2057, 1005,
1040, 2464, 1996, 3743, 1997, 1996, 2679, 2043, 1996, 5025, 2679, 2001, 2006,
1010, 2021, 2008, 2196, 3957, 1996, 4281, 1998, 3563, 2780, 1012, 1026, 7987,
1013, 1028, 1026, 7987, 1013, 1028, 2065, 2017, 3427, 2146, 2126, 2461, 2059,
2017, 2180, 1005, 1056, 2022, 4527, 2011, 1996, 2653, 2029, 19597, 1045, 2424,
2062, 19142, 2084, 5805, 1012, 1026, 7987, 1013, 1028, 1026, 7987, 1013, 1028,
1045, 2228, 1996, 2069, 2518, 2008, 11654, 2033, 2055, 1996, 4966, 2001, 4918,
1005, 1055, 2606, 1010, 2021, 2002, 2018, 2009, 13650, 2077, 4830, 6673, 2061,
2026, 12320, 2342, 2005, 15708, 2791, 2001, 4632, 6692, 5999, 1025, 27793, 1005,
1045, 2071, 2031, 2973, 2302, 1996, 1005, 8457, 1005, 6151, 3111, 8840, 2140,
2004, 2007, 1048, 13088, 1010, 1996, 7547, 2001, 2296, 2978, 2004, 5875, 2004,
1996, 2679, 2993, 1012, 1045, 3053, 6639, 2043, 4918, 3631, 2010, 2192, 1010,
1998, 15574, 2012, 2296, 24851, 2002, 8760, 2096, 2731, 1012, 1012, 1012, 1012,
1998, 1997, 2607, 1996, 2331, 1997, 5557, 10250, 14808, 14517, 1012, 1012, 1012,
2008, 2001, 2019, 10439, 8095, 2075, 10576, 1010, 2021, 2059, 2296, 2095, 2045,
1005, 1055, 2242, 1012, 1026, 7987, 1013, 1028, 1026, 7987, 1013, 1028, 18072,
9297, 2033, 12264, 1010, 2348, 2010, 7729, 2038, 5301, 1037, 4595, 2335, 2013,
1996, 6685, 8082, 22330, 8713, 2002, 2001, 1999, 1048, 13088, 1012, 2009, 1005,
1055, 2307, 2000, 2156, 2032, 2131, 2247, 2061, 2092, 2085, 2007, 4918, 1012,
1026, 7987, 1013, 1028, 1026, 7987, 1013, 1028, 2054, 1045, 4342, 2013, 2023,
18735, 2001, 1011, 1015, 1012, 2196, 2292, 22015, 7374, 2115, 4316, 2005, 2505,
999, 1011, 2027, 2018, 2706, 2000, 7374, 1996, 1060, 2629, 1010, 1998, 2145,
1996, 2154, 2077, 1996, 2136, 2187, 2005, 11929, 1010, 22015, 2018, 2069, 2589,
2431, 1997, 2477, 2008, 2734, 2000, 2022, 2589, 1010, 1998, 1996, 4316, 2001,
1037, 3255, 2802, 1996, 2878, 2679, 1025, 1016, 1012, 1996, 4830, 6673, 18829,
2342, 2000, 2404, 1037, 2843, 2062, 2147, 2046, 2037, 7945, 1013, 4062, 26384,
2933, 1011, 2975, 4717, 1006, 1998, 10712, 1037, 2312, 2193, 1997, 2060, 8195,
1013, 6853, 2041, 2000, 4318, 1996, 2126, 2027, 2106, 2001, 2498, 2460, 1997,
12731, 14277, 3085, 27988, 1025, 1017, 1012, 4918, 2038, 2019, 2203, 27242,
12024, 2005, 1005, 5931, 1998, 7823, 1005, 6172, 2021, 3791, 2000, 7823, 2368,
```



```
[13]: def tokenize(batch):
        temp = tokenizer(batch['review'], padding=True, truncation = True,
        ↪max_length=300)
        return temp

dataset = dataset.map(tokenize, batched=True, batch_size = None)
```

```
[14]: dataset['train'][0].keys()
```

4 Building Model Evaluation Functions

```
[16]: import torch
import transformers
```

```
print(torch.__version__)
print(transformers.__version__)
```

2.2.2+cu121

4.46.3

5 Model Building and Training

```
[17]: from transformers import AutoModelForSequenceClassification, TrainingArguments, \
      ↪ Trainer

model = AutoModelForSequenceClassification.from_pretrained(
    model_ckpt,
    num_labels=len(label2id),
    id2label=id2label,
    label2id=label2id,
)
```

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at huawei-noah/TinyBERT_General_4L_312D and are newly initialized: ['classifier.bias', 'classifier.weight']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

```
[ ]: from transformers import TrainingArguments

args = TrainingArguments(
    output_dir="out",
    logging_dir="logs",
    report_to="none",          # TensorBoard disable
    logging_strategy="steps",
    logging_steps=50,

    num_train_epochs=3,
    learning_rate=2e-5,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    evaluation_strategy="epoch",
    save_strategy="epoch",
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
```

```
compute_metrics=compute_metrics,  
tokenizer=tokenizer,  
)
```

```
[30]: trainer.train()
```

```
2%|          | 50/3282 [01:41<1:56:13, 2.16s/it]  
{'loss': 0.6838, 'grad_norm': 3.5912768840789795, 'learning_rate':  
1.9695307739183427e-05, 'epoch': 0.05}  
  
3%|          | 100/3282 [03:19<1:44:22, 1.97s/it]  
{'loss': 0.5772, 'grad_norm': 7.42074728012085, 'learning_rate':  
1.9390615478366852e-05, 'epoch': 0.09}  
  
5%|          | 150/3282 [04:59<1:45:39, 2.02s/it]  
{'loss': 0.4954, 'grad_norm': 5.878271579742432, 'learning_rate':  
1.9085923217550274e-05, 'epoch': 0.14}  
  
6%|          | 200/3282 [06:39<1:44:49, 2.04s/it]  
{'loss': 0.4565, 'grad_norm': 7.908751487731934, 'learning_rate':  
1.87812309567337e-05, 'epoch': 0.18}  
  
8%|          | 250/3282 [08:21<1:38:54, 1.96s/it]  
{'loss': 0.4462, 'grad_norm': 20.869808197021484, 'learning_rate':  
1.8476538695917125e-05, 'epoch': 0.23}  
  
9%|          | 300/3282 [10:00<1:38:15, 1.98s/it]  
{'loss': 0.4304, 'grad_norm': 5.636259078979492, 'learning_rate':  
1.817184643510055e-05, 'epoch': 0.27}  
  
11%|         | 350/3282 [11:38<1:35:04, 1.95s/it]  
{'loss': 0.4109, 'grad_norm': 19.33477020263672, 'learning_rate':  
1.7867154174283973e-05, 'epoch': 0.32}  
  
12%|         | 400/3282 [13:17<1:35:59, 2.00s/it]  
{'loss': 0.361, 'grad_norm': 9.618731498718262, 'learning_rate':  
1.7562461913467398e-05, 'epoch': 0.37}  
  
14%|         | 450/3282 [14:57<1:36:11, 2.04s/it]  
{'loss': 0.3827, 'grad_norm': 14.839694023132324, 'learning_rate':  
1.7257769652650823e-05, 'epoch': 0.41}  
  
15%|         | 500/3282 [16:37<1:32:13, 1.99s/it]  
{'loss': 0.3751, 'grad_norm': 7.204689025878906, 'learning_rate':  
1.695307739183425e-05, 'epoch': 0.46}  
  
17%|         | 550/3282 [18:15<1:27:56, 1.93s/it]
```



```

{'loss': 0.3553, 'grad_norm': 13.912898063659668, 'learning_rate':
1.664838513101767e-05, 'epoch': 0.5}

18%|          | 600/3282 [19:53<1:26:45, 1.94s/it]

{'loss': 0.3538, 'grad_norm': 9.410259246826172, 'learning_rate':
1.6343692870201096e-05, 'epoch': 0.55}

20%|          | 650/3282 [21:35<1:35:53, 2.19s/it]

{'loss': 0.3579, 'grad_norm': 12.562080383300781, 'learning_rate':
1.603900060938452e-05, 'epoch': 0.59}

21%|          | 700/3282 [23:16<1:24:58, 1.97s/it]

{'loss': 0.3336, 'grad_norm': 7.20489501953125, 'learning_rate':
1.5734308348567947e-05, 'epoch': 0.64}

23%|          | 750/3282 [24:54<1:24:16, 2.00s/it]

{'loss': 0.34, 'grad_norm': 17.05854034423828, 'learning_rate':
1.5429616087751372e-05, 'epoch': 0.69}

24%|          | 800/3282 [26:31<1:23:11, 2.01s/it]

{'loss': 0.364, 'grad_norm': 13.826316833496094, 'learning_rate':
1.5124923826934796e-05, 'epoch': 0.73}

26%|          | 850/3282 [28:10<1:18:07, 1.93s/it]

{'loss': 0.341, 'grad_norm': 26.642301559448242, 'learning_rate':
1.4820231566118222e-05, 'epoch': 0.78}

27%|          | 900/3282 [29:48<1:20:22, 2.02s/it]

{'loss': 0.3638, 'grad_norm': 8.851871490478516, 'learning_rate':
1.4515539305301647e-05, 'epoch': 0.82}

29%|          | 950/3282 [31:27<1:15:13, 1.94s/it]

{'loss': 0.3431, 'grad_norm': 29.811716079711914, 'learning_rate':
1.4210847044485072e-05, 'epoch': 0.87}

30%|          | 1000/3282 [33:05<1:15:24, 1.98s/it]

{'loss': 0.3383, 'grad_norm': 13.871434211730957, 'learning_rate':
1.3906154783668494e-05, 'epoch': 0.91}

32%|          | 1050/3282 [34:42<1:11:37, 1.93s/it]

{'loss': 0.3484, 'grad_norm': 7.766380786895752, 'learning_rate':
1.360146252285192e-05, 'epoch': 0.96}

33%|          | 1094/3282 [40:14<1:06:57, 1.84s/it]

{'eval_loss': 0.31005144119262695, 'eval_accuracy': 0.8662666666666666,
'eval_runtime': 247.2289, 'eval_samples_per_second': 60.673,
'eval_steps_per_second': 1.897, 'epoch': 1.0}

```

34%| | 1100/3282 [40:27<8:45:19, 14.45s/it]
{'loss': 0.3451, 'grad_norm': 5.275298595428467, 'learning_rate':
1.3296770262035345e-05, 'epoch': 1.01}

35%| | 1150/3282 [42:04<1:08:18, 1.92s/it]
{'loss': 0.3159, 'grad_norm': 15.184335708618164, 'learning_rate':
1.299207800121877e-05, 'epoch': 1.05}

37%| | 1200/3282 [43:41<1:10:38, 2.04s/it]
{'loss': 0.3054, 'grad_norm': 8.304593086242676, 'learning_rate':
1.2687385740402194e-05, 'epoch': 1.1}

38%| | 1250/3282 [45:17<1:05:05, 1.92s/it]
{'loss': 0.3157, 'grad_norm': 12.097025871276855, 'learning_rate':
1.238269347958562e-05, 'epoch': 1.14}

40%| | 1300/3282 [46:54<1:04:00, 1.94s/it]
{'loss': 0.2883, 'grad_norm': 8.152347564697266, 'learning_rate':
1.2078001218769045e-05, 'epoch': 1.19}

41%| | 1350/3282 [48:31<1:02:03, 1.93s/it]
{'loss': 0.2939, 'grad_norm': 9.285045623779297, 'learning_rate':
1.1773308957952469e-05, 'epoch': 1.23}

43%| | 1400/3282 [50:07<1:00:14, 1.92s/it]
{'loss': 0.3127, 'grad_norm': 15.291668891906738, 'learning_rate':
1.1468616697135894e-05, 'epoch': 1.28}

44%| | 1450/3282 [51:44<58:59, 1.93s/it]
{'loss': 0.3013, 'grad_norm': 6.322088718414307, 'learning_rate':
1.1163924436319318e-05, 'epoch': 1.33}

46%| | 1500/3282 [53:20<57:10, 1.92s/it]
{'loss': 0.2946, 'grad_norm': 9.099776268005371, 'learning_rate':
1.0859232175502743e-05, 'epoch': 1.37}

47%| | 1550/3282 [54:57<55:53, 1.94s/it]
{'loss': 0.3048, 'grad_norm': 9.244580268859863, 'learning_rate':
1.0554539914686169e-05, 'epoch': 1.42}

49%| | 1600/3282 [56:34<53:54, 1.92s/it]
{'loss': 0.2966, 'grad_norm': 10.026045799255371, 'learning_rate':
1.0249847653869594e-05, 'epoch': 1.46}

50%| | 1650/3282 [58:12<53:46, 1.98s/it]
{'loss': 0.2836, 'grad_norm': 7.976522922515869, 'learning_rate':
9.945155393053018e-06, 'epoch': 1.51}

52%| | 1700/3282 [59:49<54:03, 2.05s/it]
{'loss': 0.2947, 'grad_norm': 14.400959014892578, 'learning_rate':
9.640463132236442e-06, 'epoch': 1.55}

53%| | 1750/3282 [1:01:27<49:12, 1.93s/it]
{'loss': 0.276, 'grad_norm': 8.11874771118164, 'learning_rate':
9.335770871419867e-06, 'epoch': 1.6}

55%| | 1800/3282 [1:03:05<47:31, 1.92s/it]
{'loss': 0.2724, 'grad_norm': 11.260644912719727, 'learning_rate':
9.03107861060329e-06, 'epoch': 1.65}

56%| | 1850/3282 [1:04:42<45:55, 1.92s/it]
{'loss': 0.2827, 'grad_norm': 15.130202293395996, 'learning_rate':
8.726386349786716e-06, 'epoch': 1.69}

58%| | 1900/3282 [1:06:19<44:28, 1.93s/it]
{'loss': 0.2665, 'grad_norm': 13.282217025756836, 'learning_rate':
8.42169408897014e-06, 'epoch': 1.74}

59%| | 1950/3282 [1:07:57<42:54, 1.93s/it]
{'loss': 0.2873, 'grad_norm': 7.424344062805176, 'learning_rate':
8.117001828153565e-06, 'epoch': 1.78}

61%| | 2000/3282 [1:09:35<41:13, 1.93s/it]
{'loss': 0.3059, 'grad_norm': 13.0115385055542, 'learning_rate':
7.81230956733699e-06, 'epoch': 1.83}

62%| | 2050/3282 [1:11:12<39:36, 1.93s/it]
{'loss': 0.2928, 'grad_norm': 6.654662132263184, 'learning_rate':
7.507617306520415e-06, 'epoch': 1.87}

64%| | 2100/3282 [1:12:50<38:20, 1.95s/it]
{'loss': 0.2769, 'grad_norm': 4.29531192779541, 'learning_rate':
7.2029250457038405e-06, 'epoch': 1.92}

66%| | 2150/3282 [1:14:28<36:23, 1.93s/it]
{'loss': 0.2666, 'grad_norm': 20.795663833618164, 'learning_rate':
6.898232784887264e-06, 'epoch': 1.97}

67%| | 2188/3282 [1:19:46<32:48, 1.80s/it]
{'eval_loss': 0.3009887933731079, 'eval_accuracy': 0.8766, 'eval_runtime':
244.4946, 'eval_samples_per_second': 61.351, 'eval_steps_per_second': 1.918,
'epoch': 2.0}

67%| | 2200/3282 [1:20:11<1:02:38, 3.47s/it]

```

{'loss': 0.2975, 'grad_norm': 12.683072090148926, 'learning_rate':
6.59354052407069e-06, 'epoch': 2.01}

69%|      | 2250/3282 [1:21:48<33:06, 1.92s/it]

{'loss': 0.2692, 'grad_norm': 6.658638954162598, 'learning_rate':
6.288848263254113e-06, 'epoch': 2.06}

70%|      | 2300/3282 [1:23:25<31:32, 1.93s/it]

{'loss': 0.2655, 'grad_norm': 21.7436580657959, 'learning_rate':
5.984156002437539e-06, 'epoch': 2.1}

72%|      | 2350/3282 [1:25:02<30:05, 1.94s/it]

{'loss': 0.2735, 'grad_norm': 12.682290077209473, 'learning_rate':
5.679463741620963e-06, 'epoch': 2.15}

73%|      | 2400/3282 [1:26:40<28:45, 1.96s/it]

{'loss': 0.2387, 'grad_norm': 10.242602348327637, 'learning_rate':
5.374771480804388e-06, 'epoch': 2.19}

75%|      | 2450/3282 [1:28:18<27:27, 1.98s/it]

{'loss': 0.2555, 'grad_norm': 14.095138549804688, 'learning_rate':
5.0700792199878124e-06, 'epoch': 2.24}

76%|      | 2500/3282 [1:29:55<26:37, 2.04s/it]

{'loss': 0.2544, 'grad_norm': 12.122700691223145, 'learning_rate':
4.765386959171238e-06, 'epoch': 2.29}

78%|      | 2550/3282 [1:31:37<25:02, 2.05s/it]

{'loss': 0.2475, 'grad_norm': 9.858541488647461, 'learning_rate':
4.460694698354662e-06, 'epoch': 2.33}

79%|      | 2600/3282 [1:33:17<22:25, 1.97s/it]

{'loss': 0.2507, 'grad_norm': 14.106688499450684, 'learning_rate':
4.156002437538087e-06, 'epoch': 2.38}

81%|      | 2650/3282 [1:34:58<21:45, 2.07s/it]

{'loss': 0.2589, 'grad_norm': 8.371150970458984, 'learning_rate':
3.8513101767215115e-06, 'epoch': 2.42}

82%|      | 2700/3282 [1:36:37<18:50, 1.94s/it]

{'loss': 0.2435, 'grad_norm': 34.76189041137695, 'learning_rate':
3.5466179159049365e-06, 'epoch': 2.47}

84%|      | 2750/3282 [1:38:16<17:12, 1.94s/it]

{'loss': 0.2641, 'grad_norm': 17.966638565063477, 'learning_rate':
3.241925655088361e-06, 'epoch': 2.51}

85%|      | 2800/3282 [1:39:59<16:18, 2.03s/it]

```

```

{'loss': 0.2648, 'grad_norm': 9.304388046264648, 'learning_rate':
2.9372333942717856e-06, 'epoch': 2.56}

87%|      | 2850/3282 [1:41:42<14:44, 2.05s/it]

{'loss': 0.2513, 'grad_norm': 5.547393321990967, 'learning_rate':
2.63254113345521e-06, 'epoch': 2.61}

88%|      | 2900/3282 [1:43:26<13:17, 2.09s/it]

{'loss': 0.2475, 'grad_norm': 6.8576436042785645, 'learning_rate':
2.327848872638635e-06, 'epoch': 2.65}

90%|      | 2950/3282 [1:45:09<11:33, 2.09s/it]

{'loss': 0.2775, 'grad_norm': 9.137205123901367, 'learning_rate':
2.02315661182206e-06, 'epoch': 2.7}

91%|      | 3000/3282 [1:46:55<10:06, 2.15s/it]

{'loss': 0.2554, 'grad_norm': 5.975708961486816, 'learning_rate':
1.7184643510054846e-06, 'epoch': 2.74}

93%|      | 3050/3282 [1:48:40<08:14, 2.13s/it]

{'loss': 0.2362, 'grad_norm': 10.277643203735352, 'learning_rate':
1.4137720901889094e-06, 'epoch': 2.79}

94%|      | 3100/3282 [1:50:25<06:27, 2.13s/it]

{'loss': 0.2407, 'grad_norm': 8.855224609375, 'learning_rate':
1.109079829372334e-06, 'epoch': 2.83}

96%|      | 3150/3282 [1:52:08<04:42, 2.14s/it]

{'loss': 0.2401, 'grad_norm': 11.408275604248047, 'learning_rate':
8.043875685557588e-07, 'epoch': 2.88}

98%|      | 3200/3282 [1:53:52<02:55, 2.14s/it]

{'loss': 0.246, 'grad_norm': 22.325843811035156, 'learning_rate':
4.996953077391835e-07, 'epoch': 2.93}

99%|      | 3250/3282 [1:55:36<01:07, 2.10s/it]

{'loss': 0.2505, 'grad_norm': 14.014134407043457, 'learning_rate':
1.950030469226082e-07, 'epoch': 2.97}

100%|     | 3282/3282 [2:01:53<00:00, 2.01s/it]

{'eval_loss': 0.297394335269928, 'eval_accuracy': 0.8812666666666666,
'eval_runtime': 309.7716, 'eval_samples_per_second': 48.423,
'eval_steps_per_second': 1.514, 'epoch': 3.0}

100%|     | 3282/3282 [2:01:55<00:00, 2.23s/it]

{'train_runtime': 7315.6331, 'train_samples_per_second': 14.353,
'train_steps_per_second': 0.449, 'train_loss': 0.3159478267179183, 'epoch': 3.0}

```

```
[30]: TrainOutput(global_step=3282, training_loss=0.3159478267179183,
metrics={'train_runtime': 7315.6331, 'train_samples_per_second': 14.353,
'train_steps_per_second': 0.449, 'total_flos': 882184338000000.0, 'train_loss':
0.3159478267179183, 'epoch': 3.0})
```

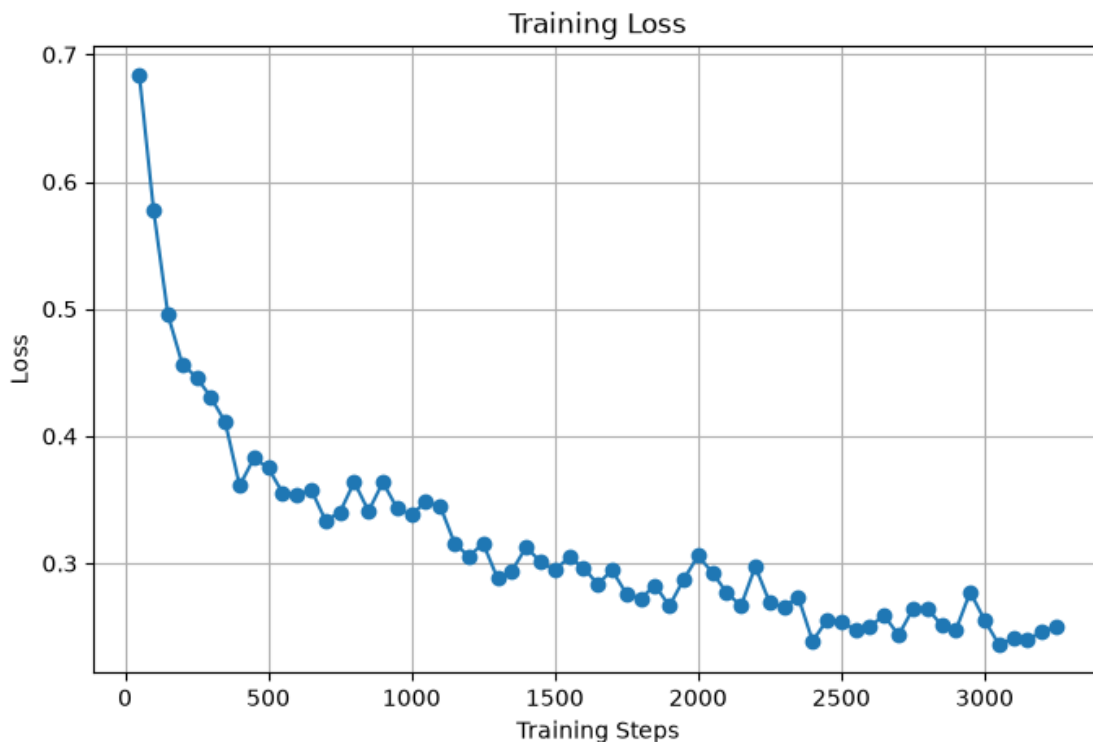
```
[33]: import matplotlib.pyplot as plt

logs = trainer.state.log_history

train_loss = []
steps = []

for log in logs:
    if "loss" in log:
        train_loss.append(log["loss"])
        steps.append(log["step"])

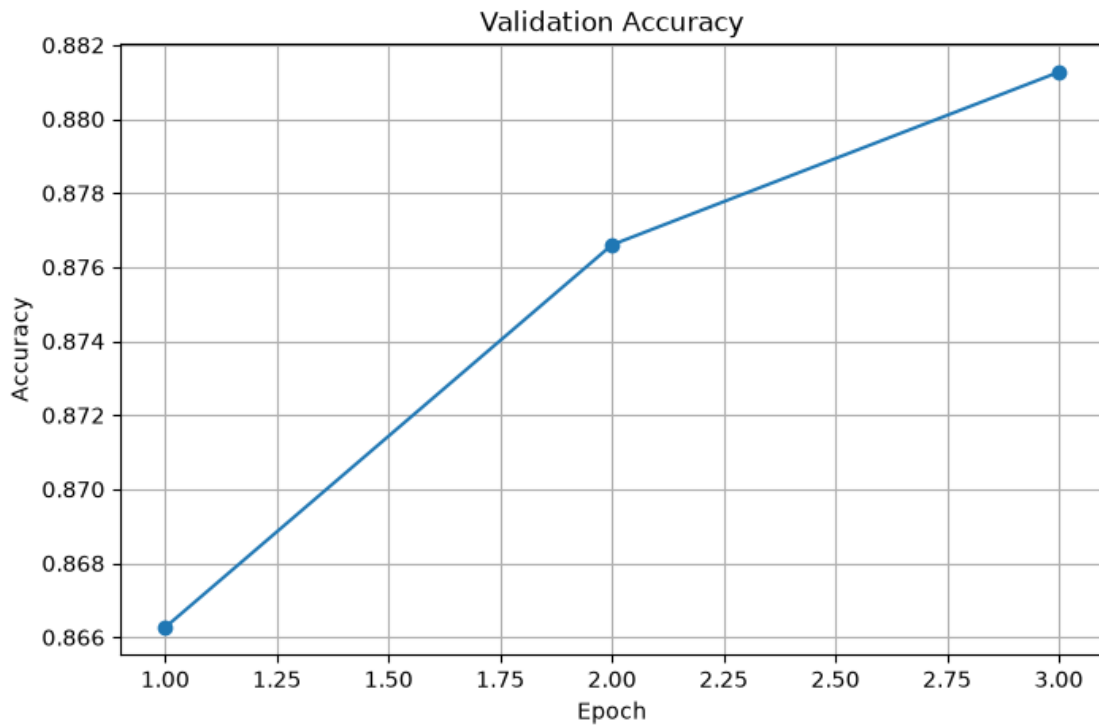
plt.figure(figsize=(8,5))
plt.plot(steps, train_loss, marker="o")
plt.title("Training Loss")
plt.xlabel("Training Steps")
plt.ylabel("Loss")
plt.grid(True)
plt.show()
```



```
[34]: eval_acc = []
epochs = []

for log in trainer.state.log_history:
    if "eval_accuracy" in log:
        eval_acc.append(log["eval_accuracy"])
        epochs.append(log["epoch"])

plt.figure(figsize=(8,5))
plt.plot(epochs, eval_acc, marker="o")
plt.title("Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.grid(True)
plt.show()
```



```
[35]: train_epochs = []
train_loss = []
eval_epochs = []
eval_loss = []
```

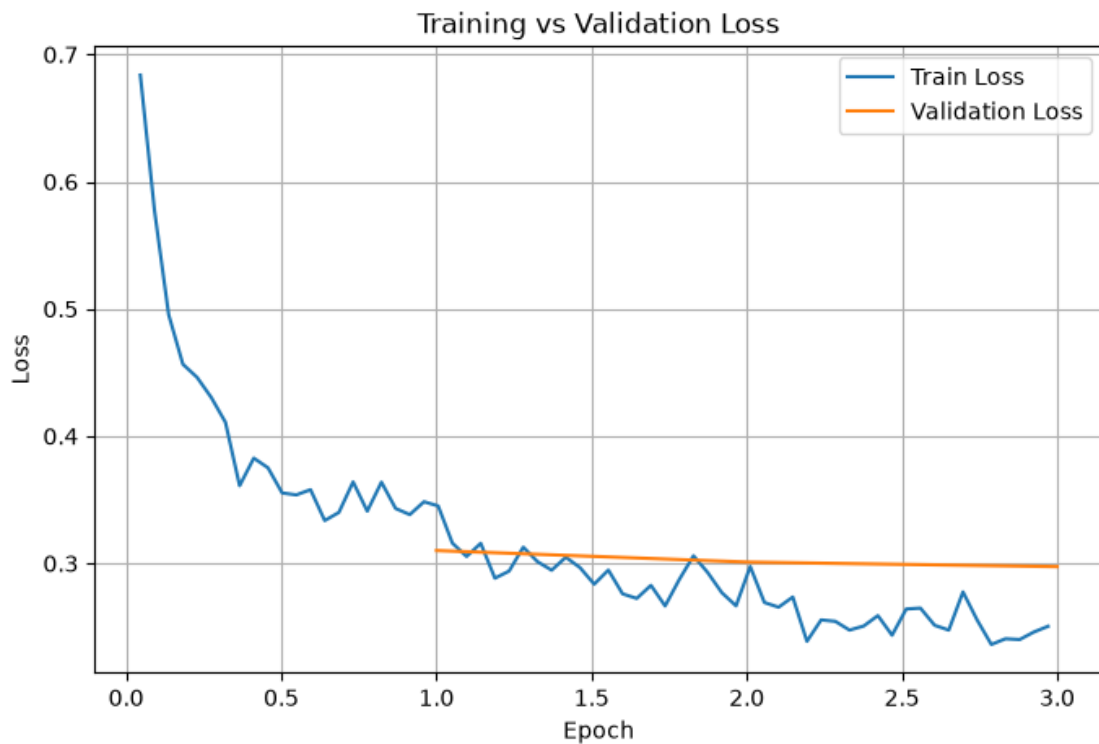
```

for log in trainer.state.log_history:
    if "loss" in log and "epoch" in log:
        train_epochs.append(log["epoch"])
        train_loss.append(log["loss"])

    if "eval_loss" in log:
        eval_epochs.append(log["epoch"])
        eval_loss.append(log["eval_loss"])

plt.figure(figsize=(8,5))
plt.plot(train_epochs, train_loss, label="Train Loss")
plt.plot(eval_epochs, eval_loss, label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.grid(True)
plt.show()

```



```
[31]: history = pd.DataFrame(trainer.state.log_history)
```



```
print(history.columns)
history.head()
```

```
Index(['loss', 'grad_norm', 'learning_rate', 'epoch', 'step', 'eval_loss',
      'eval_accuracy', 'eval_runtime', 'eval_samples_per_second',
      'eval_steps_per_second', 'train_runtime', 'train_samples_per_second',
      'train_steps_per_second', 'total_flos', 'train_loss'],
      dtype='str')
```

```
[31]:
```

	loss	grad_norm	learning_rate	epoch	step	eval_loss	eval_accuracy	\
0	0.6838	3.591277	0.000020	0.045704	50	NaN	NaN	
1	0.5772	7.420747	0.000019	0.091408	100	NaN	NaN	
2	0.4954	5.878272	0.000019	0.137112	150	NaN	NaN	
3	0.4565	7.908751	0.000019	0.182815	200	NaN	NaN	
4	0.4462	20.869808	0.000018	0.228519	250	NaN	NaN	

	eval_runtime	eval_samples_per_second	eval_steps_per_second	\
0	NaN	NaN	NaN	
1	NaN	NaN	NaN	
2	NaN	NaN	NaN	
3	NaN	NaN	NaN	
4	NaN	NaN	NaN	

	train_runtime	train_samples_per_second	train_steps_per_second	\
0	NaN	NaN	NaN	
1	NaN	NaN	NaN	
2	NaN	NaN	NaN	
3	NaN	NaN	NaN	
4	NaN	NaN	NaN	

	total_flos	train_loss
0	NaN	NaN
1	NaN	NaN
2	NaN	NaN
3	NaN	NaN
4	NaN	NaN

5.1 Model Save and Load for Inference

```
[ ]: trainer.save_model('tinybert_sentiment_analysis')
```

5.2 Using the Trained Model for the Prediction

```
[12]: text = ["The cast gave brilliant, career-defining performances.",
              "However, the confusing script ruined the movie.",
              "I wanted to love this highly anticipated sequel.",
              "Sadly, the abrupt ending left me frustrated.",
              "While the cinematography is breathtakingly gorgeous throughout,"]
```

```
"the structural narrative falls completely flat ultimately."]
```

```
[3]: import torch
device = torch.device('cuda') if torch.cuda.is_available() else torch.
↳device('cpu')
```

```
[4]: from transformers import pipeline
classifier = pipeline('text-classification', model =_
↳'tinybert_sentiment_analysis', device = device)
```

```
c:\Users\Uditya\anaconda3\envs\genai\Lib\site-packages\tqdm\auto.py:21:
TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
```

```
[13]: df = classifier(text)
```

```
[16]: import pandas as pd
data = pd.DataFrame(df)
data['Review'] = text
data['Predicted Sentiment Score'] = data['score']*100
data['Predicted Label'] = data['label']
data = data.drop(['label', 'score'], axis = 1)
data
```

```
[16]:
```

	Review \
0	The cast gave brilliant, career-defining perfo...
1	However, the confusing script ruined the movie.
2	I wanted to love this highly anticipated sequel.
3	Sadly, the abrupt ending left me frustrated.
4	While the cinematography is breathtakingly gor...
5	the structural narrative falls completely flat...

	Predicted Sentiment Score	Predicted Label
0	98.900628	positive
1	99.029517	negative
2	97.028154	positive
3	97.957176	negative
4	98.954481	positive
5	98.374647	negative

```
[ ]: data.to_csv('tinybert_sentiment_results.csv', index=False)
```

5.3 Push Model to AWS S3

```
[24]: import boto3

s3 = boto3.client('s3')

bucket_name = 'sentiment-prediction-model-tinybert'

def create_bucket(bucket_name):
    response = s3.list_buckets()
    buckets = [buck['Name'] for buck in response['Buckets']]
    if bucket_name not in buckets:
        s3.create_bucket(Bucket=bucket_name)
        print("Bucket is Created")
    else:
        print("Bucket already exists in your account!!! Fell free to use it..")

create_bucket(bucket_name)
```

Bucket already exists in your account!!! Fell free to use it..

5.4 Uploading the Model Folder to S3 Bucket sentiment-prediction-model-tinybert/tinybert_sentiment_analysis

```
[26]: import os

import boto3
s3 = boto3.client('s3')
bucket_name = 'sentiment-prediction-model-tinybert'

def upload_directory(directory_path, s3_prefix):
    for root, dirs, files in os.walk(directory_path):
        for file in files:
            file_path = os.path.join(root, file).replace("\\", "/")
            relpath = os.path.relpath(file_path, directory_path)
            s3_key = os.path.join(s3_prefix, relpath).replace("\\", "/")

            s3.upload_file(file_path, bucket_name, s3_key)

upload_directory('tinybert_sentiment_analysis',
    ↪ 'sentiment-prediction-model-tinybert/tinybert_sentiment_analysis')
```

```
[2]: from IPython.display import Image, display

display(Image(filename="s3upload.png"))
```

Amazon S3

▼ Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

▼ Files

File systems

▼ Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

▼ Storage management and insights

Storage Lens

Batch Operations

Account and organization settings

AWS Marketplace for S3

Amazon S3 > Buckets > sentiment-prediction-model-tinybert > sentiment-prediction-model-tinybert/ > tinybert_sentiment_analysis/

15

Objects Properties

Objects (7)

Copy S3 URI Copy URL Download Open i Delete Actions Create folder Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	config.json	json	August 5, 2026, 11:23:04 (UTC+05:30)	941.0 B	Standard
☐	model.safetensors	safetensors	August 5, 2026, 11:23:05 (UTC+05:30)	54.8 MB	Standard
☐	special_tokens_map.json	json	August 5, 2026, 11:24:00 (UTC+05:30)	132.0 B	Standard
☐	tokenizer_config.json	json	August 5, 2026, 11:24:06 (UTC+05:30)	1.3 KB	Standard
☐	tokenizer.json	json	August 5, 2026, 11:24:03 (UTC+05:30)	695.0 KB	Standard
☐	training_args.bin	bin	August 5, 2026, 11:24:07 (UTC+05:30)	5.1 KB	Standard
☐	vocab.txt	txt	August 5, 2026, 11:24:08 (UTC+05:30)	226.1 KB	Standard

20